

ACRME — Full Project Research & Design Prompt

Document Type: Master Research & Design Prompt\

Intended For: AI assistants, new architects, platform engineers, and technical researchers picking up this project cold\

Baseline: Azure Capacity & Quota Management Consolidated Requirements Baseline v2.4 (amended)\

Repo: vishnu4573/acrme-capacity-reservation (branch: master)\

Last updated: 6 October 2026

How to Use This Prompt

Read the entire prompt before asking any questions or producing any outputs. It is written in two layers:

1. **Context Layer (Sections 1-6):** What the project is, what problem it solves, and what has already been decided. Read this to build a grounded mental model.
2. **Task Layer (Section 7):** The specific research or design work you are being asked to do. Your task is defined there.

Source of truth hierarchy:

1. Microsoft Learn / Azure documentation (supersedes everything)
2. Azure Blogs, Product Group Guidance, Azure Updates, Microsoft GitHub Samples
3. Internal project documents (repo `Reference-Material/` , `Design/` , `Requirements/`)
4. Community sources (only when official guidance does not exist)

Confidence tagging — every claim you produce must carry one of these labels:

- `[Documented]` — backed by Microsoft Learn or Azure platform documentation
- `[Decided]` — an approved design decision recorded in the project ADR log
- `[Derived]` — a logical consequence of the design; requires POC validation where noted
- `[Assumed]` — a policy default or working hypothesis; not yet empirically validated

Never produce a recommendation without identifying its confidence level. Distinguish clearly between what is confirmed and what needs a proof-of-concept (POC).

Section 1 — Project Identity

What ACRME Is

ACRME (Azure Capacity Reservation Management Engine) is a **SaaS platform capacity governance engine** built on top of Azure VM Capacity Reservations and Azure VM Quota. It manages capacity and quota for a multi-region, multi-environment production SaaS platform — protecting deployment capacity for production workloads, governing DR reserves, automating quota pooling, and feeding placement decisions into the AEP (Azure Engineering Platform) provisioning workflow.

What Problem It Solves

Without ACRME, the platform faces three compounding risks:

1. **Deployment failures at scale** — Azure does not guarantee VM capacity on-demand. Without capacity reservations, Prod deployments can fail because the required SKU is unavailable in the target region at deployment time.
2. **Uncontrolled DR cost** — Fixed large DR reserves are estimated in the millions of dollars per year in idle reservation cost. ACRME replaces fixed large reserves with a minimal bootstrap that dynamically scales on demand.
3. **Quota exhaustion at deployment time** — Regional VM quota is shared across hundreds of subscriptions. Without centralized governance, individual teams can exhaust quota, blocking deployments silently. ACRME treats quota as a centrally governed resource (“quota-as-governor” strategy per Melvin Stephen, CVP).

What ACRME Is Not

- It is **not a multi-cloud solution**. Multi-cloud DR is explicitly out of scope (rejected at ELT level).
- It is **not a VM scheduler or orchestrator**. It manages the reservation and quota envelope, not the deployment pipeline itself.
- It is **not static or configuration-fixed**. It is designed to be configuration-driven because region scope, DR decisions, and customer commitments change frequently.

Section 2 — The Azure Primitives ACRME Operates On

Understanding these Azure concepts is prerequisite to any design or research on this project.

2.1 Azure VM Capacity Reservation

A **Capacity Reservation** is a commitment to hold a specific quantity of a specific VM SKU in a specific Azure region and Availability Zone. Key facts:

- [Documented] Reserved capacity is billed whether or not VMs are deployed against it.
- [Documented] Capacity Reservations are zonal by default (tied to an Availability Zone). A no-zone group is pinned by Azure to one zone it selects.
- [Documented] Only VMs deployed in an Availability Zone (Zonal VMs) are eligible. VMs in Availability Sets are **ineligible** (CAP-020).

- [Documented] VMs must be deallocated or migrated to a zone before they can be associated (CAP-021).
- [Documented] Deallocated VMs that are still associated to a reservation continue to hold quota until explicitly dissociated (CAP-004).
- [Decided] CRG naming convention: `crg-{geo}-{region_short}-{env}-{az}` for per-AZ groups; `crg-{geo}-{region_short}-{env}-reg` for regional groups (OPS-006, C-12).

2.2 Capacity Reservation Group (CRG)

A **Capacity Reservation Group** is the container that holds one or more Capacity Reservations and the VMs associated with them.

- [Decided] Structure: **regional CRG** (`crg-...-reg`) for most reservation management + **per-AZ CRGs** (`crg-...-az1/az2/az3`) for zone-level capacity (CAP-023).
- [Decided] The managed SKU/AZ matrix is seeded at zero (count=0) when a region is onboarded, then reconciled reactively (CAP-022, CAP-024).
- [Preview – unvalidated] **Capacity Reservation Sharing** (the ability for VMs in one subscription to consume a reservation owned by a different subscription) is a **Microsoft Preview feature**. The top technical unknown (POC-001) is whether the consumer subscription must hold its own quota when consuming a shared reservation.

2.3 Azure VM Quota

Quota is a regional cap on how many vCPUs of a given VM family a subscription can deploy.

- [Documented] Quota has two caps: a **VM family cap** (e.g., “ESv5 family — max 1000 vCPUs in East US 2”) and a **regional total vCPU cap** across all families.
- [Documented] A **Quota Group** is an Azure mechanism for transferring quota between subscriptions under the same Entra ID tenant.
- [Decided] ACRME uses a **single governed quota pool per region per VM family**, shared across Prod, CVAL/NonProd, and DR (QUA-004). Prod and DR capacity are protected by logical earmarks within this pool, not by separate Azure quota groups (except as a narrow governance exception).
- [Decided] The consumer subscription must hold its own quota at deployment time — sharing does not transfer quota (QUA-013, resolved from POC-001; [Documented] per Microsoft Learn corrections in v2.5).

Section 3 — Architecture: The Three Domains

ACRME operates across three tightly coupled domains. Any design or research that touches one typically has implications for the others.

3.1 Domain 1 — Region Selection (Placement Scoring)

Before any customer is deployed, ACRME scores candidate regions using a weighted placement formula and selects the best available region per environment. This is the **output interface** to AEP provisioning.

Two input paths:

- **Default (customer supplies exact region):** ACRME validates the region against hard constraints (HC-1..HC-10) and computes `PS_Prod` as a post-validation readiness check.
- **Exception (customer picks a geography only):** ACRME runs `argmax(PS_Prod)` over in-scope Standard regions in that geography, with explicit approval + customer acknowledgement required.

Placement Score formula (Prod):

```
PS_Prod(r) = α·Clamp(nonprod_crg.effective_free / prod_crg.quantity)
            + β·Clamp(prod_crg.quota_headroom / prod_crg.quota_limit)
            + γ·(1 - prod_customer_count / total_customers)      [total_customers:
UNDEFINED in v2.4 - spec gap]
            + δ·dr_crg.coverage_ratio
            + ε·(az_count / 3)
```

Default weights: $\alpha=0.30$, $\beta=0.20$, $\gamma=0.25$, $\delta=0.15$, $\epsilon=0.10$. All components clamped to [0,1]. Weights sum to 1.00.

Critical design decision — why α uses NonProd headroom (not Prod headroom):\

Using Prod headroom in α would be circular (measuring the thing you're about to allocate against itself) and would create high correlation between α and β (combined 0.50 weight measuring the same Prod dimension). NonProd headroom is an independent signal of regional capacity health and provides forward-looking overflow/failover capacity. See `Reference-Material/reference/ps_prod_nonprod_headroom_design_rationale.md` for the full worked rationale.

total_customers is an open specification gap. The γ term references `total_customers` which has no defined source, scope, or data type in baseline v2.4. This is a blocker for γ in production scoring. Flag this gap in any design output.

Environment-specific PS functions: `PS_NonProd` and `PS_DR` use the same weight structure with the relevant environment's CRG metrics substituted.

3.2 Domain 2 — Quota Management

Quota is managed as a central, pooled resource governed across all subscriptions. The engine:

- Maintains a single quota pool per region per VM family (QUA-004)
- Tracks `Available Quota = Assigned Regional VM-Family Quota - Current Regional VM-Family Usage`
- Manages a production quota buffer for growth (POC-008 defines initial buffer targets)
- Enforces DR floor accounting (`DR_Floor_vCPU`) to protect DR allocation
- Triggers emergency quota transfers and tier escalations when thresholds are breached

Key formula:

```
Quota Deficit = max(0, Requested Deployment Units - Available Quota)
```

3.3 Domain 3 — Capacity Reservation Management

The engine manages the lifecycle of Capacity Reservations: creation, reconciliation, growth, scale-down, and decommissioning.

Reservation target (CAP-003):

```
Target Reserved Capacity = Allocated VM Count + Buffer Target
Reservation Headroom = Reserved Quantity - Allocated VM Count
Reservation Deficit = max(0, Target Reserved Capacity - Reserved Quantity)
```

Reconciliation: The engine continuously reconciles Reserved Quantity toward Target Reserved Capacity. Reconciliation is dynamic — scale-up is reactive to growth; scale-down is governed (not automatic, CAP-008/CAP-010).

Seed-at-zero matrix (CAP-022): When a region is onboarded, the eligible-SKU/AZ matrix is seeded at count=0. Product teams must approve budget before quantity is increased.

Reactive discovery (CAP-024): New SKUs or AZ combinations are auto-discovered and added to the matrix, subject to CAP-019 governance approval.

Section 4 — Region Model & Distribution Topology

Five geographies are in scope. The distribution model differs by geography.

4.1 US — Three-Region Model

Region	Role	Notes
West US 3	Standard	Full Prod/CVAL/DR candidate
Central US	Standard	Full Prod/CVAL/DR candidate
Canada Central	Standard	Full Prod/CVAL/DR candidate
East US 2	Restricted	Exception-only (REG-003)

Prod, CVAL, and DR each score independently across the three Standard regions. DR uses distributed-portion sizing (max-not-sum, A.6).

4.2 Europe, Australia, Asia Pacific — Two-Region Co-Located Model

In every two-region geography, **CVAL and DR co-locate in the non-production region** (PLC-010a):

- Prod → Region A (weighted selected)
- CVAL + DR → Region B (the other Standard region)

Geography	Standard Regions	Restricted
Europe	Switzerland North + Sweden Central	North Europe, West Europe
Australia	Australia East + Australia Southeast	—
Asia Pacific	East Asia + Southeast Asia	Japan East (pending confirmation)

4.3 Middle East — Cross-Geography DR Model (DEC-001, resolved v2.4)

- Prod + CVAL → co-located in a **weighted-selected Middle East** Standard region
- DR → placed in a **weighted-selected Europe Standard** region (cross-geo, DR-020, PLC-010b)
- Per-country `DR_NOT_OFFERED` flag (DR-014) retained for residual legal carve-outs
- Data-residency clearance required for applicable customer classes (assumption A-ME1)

Middle East Regions	Standard
UAE North	Standard
Saudi Arabia Central	Standard

Section 5 — DR Capacity Model

5.1 Bootstrap + Dynamic Reconciliation

DR capacity starts with a **minimal bootstrap** (POC-007 defines minimum per product) and scales on demand. Fixed large DR reserves have been rejected for cost reasons (Appendix D). `[Decided]`

- **No fixed percentage of Prod.** DR must not default to a fixed 30–40% copy of production (ENV-005). That model is **retired** (Calc Logic Reference Scenario 15 → 17; POC workbook A-09). `[Decided]`
- **Bootstrap is a configurable target**, not a ratio: a node count, SKU-specific quantity, vCPU value, workload tier, or approved percentage, set by product and region (DR-007; C-1 “Configurable — no fixed value”). It holds only the minimum needed to bring up control planes and recovery orchestration first (DR-008) — “whatever is needed to bootstrap and is used = effectively free”. `[Decided]`
- **Beyond bootstrap, capacity is acquired in stages** on activation: bootstrap headroom → existing reservation/quota in the destination → CVAL sacrifice → sharing/reassignment → pooled quota → Azure request (DR-006, DR-019). `[Decided]`
- **Destination sizing is max-not-sum** (DR-017, A.6, §5.2 below); a SUM override exists only where a contract requires concurrent-failure protection (C-11). `[Decided]`

5.2 Distributed DR — Max-Not-Sum Sizing (DR-017, Appendix A.6)

Under the single-region-failure assumption (DR-001), destination `d` never needs to host more than one source region's failover simultaneously. The DR requirement is therefore:

$$\text{Destination DR Requirement}(d) = \text{MAX over each non-concurrent source region } s \text{ (Workload portion of } s \text{ assigned to destination } d)$$

NOT the sum. A conservative SUM override (C-11) is available only where a customer contract explicitly requires protection against concurrent failures.

5.3 Reciprocal Multi-Source Hosting (DR-016)

A destination region can host DR failover for multiple source regions simultaneously (at max-not-sum sizing). The overcommit ratio:

$$\text{Overcommit Ratio}(d) = \text{SUM}(\text{source portions on } d) / \text{MAX}(\text{source portions on } d)$$

A ratio > 1 quantifies the capacity saved relative to a sum-sized reserve.

5.4 Standby Activation (DR-019)

DR failover proceeds in staged waves: `associated` → `allocated`. Standby VMs are pre-associated to reservations, then activated (powered on, quota consumed) during a DR event. POC-011 must validate that shared/overcommitted DR reservations behave correctly under standby activation.

Section 6 — Open POCs and Specification Gaps

These items are **unresolved** in baseline v2.4. Any design output must flag them explicitly rather than assuming a resolution.

Critical Open POCs

ID	What Needs Validation
POC-001	~~Quota behaviour for shared reservations — does the consumer subscription need its own quota?~~ Resolved by v2.5 Microsoft Learn corrections: consumer must hold its own quota (QUA-013, [Documented]).
POC-002	Sharing consumption order — how Azure allocates shared capacity when multiple consumers request the same SKU simultaneously
POC-003	Zone/subscription boundary combinations supported by the current preview feature version
POC-004	Safe reconciliation + retry during Azure API throttling
POC-005	VM reservation/guarantee behaviour across allocated/stopped/deallocated/associated/disassociated states
POC-006	Dedicated DR subscription + cluster vs shared production subscription topology
POC-007	Minimum bootstrap sizing per product (including required control-plane nodes and SKUs)
POC-011	Max-not-sum overcommit safety — validate shared/overcommitted DR reservations behave correctly under standby activation

Open Specification Gaps

Gap	Impact
<code>total_customers</code> undefined	γ term in PS_Prod/PS_NonProd/PS_DR is un-computable. Scoring is partial until this is defined.
Capacity Reservation Sharing is Preview	Production use requires GA confirmation or explicit Microsoft supportability agreement (DEP-001)
DR drill duration and failback (DEC-002)	Extended run vs earlier failback not yet decided
Geography exception approver (DEC-003)	Binding customer acknowledgement process not defined
Japan East (Asia Pacific)	Pending confirmation as in-scope Standard region

Section 7 — Delivery Phases

ACRME is structured in four sequential delivery phases. Design outputs must be tagged to the applicable phase.

Phase 1 — Capacity Visibility & Production Protection

Read-only capacity/quota inventory · managed scope file · production reservation reconciliation (`AL-located + Buffer`) · missing-resource validation · alerts, audit logging, dashboards, dry-run · AEP readiness API. **Objective: protect production first.**

Phase 2 — Quota Pool Automation

Quota-group inventory · dynamic allocation and reclamation · production growth buffer · quota-request workflow · correlated quota-reservation readiness. **Objective: treat quota as a centrally governed resource (quota-as-governor).**

Phase 3 — Placement & Customer Seed

Exact production-region input · seed-record creation and reuse · CVAL/DR weighted placement · **CVAL/DR co-location (PLC-010)** · concurrent-deployment controls · capacity commitment workflow. **Objective: deterministic customer placement.**

Phase 4 — Distributed DR Capacity Management

Destination workload distribution · **source→destination DR index (DR-018)** · **reciprocal multi-source hosting (DR-016)** · **max-not-sum sizing (DR-017)** · CVAL release modelling · bootstrap and recovery waves · **standby activation (DR-019)** · DR declaration workflow · capacity sharing + DR quota allocation (post-POC) · DR simulation and compliance evidence. **Objective: enterprise-scale DR capacity governance.**

Section 8 — What-If Mockups (Current Feasibility Work)

Two Excel what-if models have been built to validate the scoring formulas against synthetic data. These are **feasibility only** — they do not modify the v2.4 baseline and are not connected to live Azure state.

Phase 1 What-If — Region Selection

(`acrme_region_selection_whatif_acrme_regions.xlsx`)

- Evaluates PS_Prod, PS_NonProd, PS_DR across 8 candidate regions using blended capacity/quota numbers
- Produces Prod/CVAL/DR recommendations and a ranked candidate table
- Builder: `Mockups/build_acrme_whatif_acrme_regions.py`

Phase 2 What-If — SKU-Level Placement

(`acrme_sku_placement_whatif.xlsx`)

- Evaluates placement at **per-SKU grain** (CAP-022/CAP-023): `GROUP_AVAIL = MIN(reserved-free, family quota-available)`
 - Supports **up to 8 concurrent SKU lines per run** (multi-SKU workload)
 - **Meets-All gate**: a region is eligible only when every active SKU line has `GROUP_AVAIL ≥ Line Cores`
 - $\alpha/\beta/\epsilon$ each aggregated as **MIN across active lines** (bottleneck semantics — the tightest SKU drives the score)
 - **6-state RDY-002 readiness hierarchy**: `NO_REQUEST → RESERVATION_DEFICIT → QUOTA_DEFICIT → CAPACITY_DEFICIT → READY_WITH_RISK → READY`
 - `risk_threshold = 0.10` : `READY_WITH_RISK` fires when `MIN( $\alpha_i$ ) < 0.10`
 - Builder: `Mockups/build_acrme_sku_scoring_whatif.py`
-

Section 9 — Key Repository Artefacts

All documents are under `vishnu4573/acrme-capacity-reservation` (branch: `master`).

Path	What It Is
Requirements/ac-rme_requirements_baseline_v2_4.md	Living requirements baseline — read before any design change
Reference-Material/reference/ac-rme_calculation_logic_reference.md	Every formula, tagged [Documented/Decided/Derived/Assumed]
Reference-Material/reference/ac-rme_plain_english_walkthrough.md	Plain-English companion to the formula reference
Reference-Material/reference/ps_prod_nonprod_headroom_design_rationale.md	Design rationale for why PS_Prod α uses NonProd headroom
Design/ac-rme_sku_level_capacity_quota_design.md	SKU-level capacity & quota design doc (Phase 2 grain)
Mockups/acrme_sku_placement_whatif.xlsx	Phase 2 what-if workbook (multi-SKU, 8 lines)
Mockups/ac-rme_sku_placement_whatif_user_guide.md	User guide for Phase 2 what-if
Mockups/ac-rme_region_selection_whatif_acrme_regions.xlsx	Phase 1 what-if workbook (region-level blended scoring)
Mockups/ac-rme_region_selection_whatif_acrme_regions_user_guide.md	User guide for Phase 1 what-if

Section 10 — Design Principles (Non-Negotiable)

These principles govern every design decision in ACRME. Any design output that violates one of these requires an explicit ADR and approval.

1. **Production first.** Prod capacity reservations are never voluntarily released without an approved workflow. Prod deployment failures are the worst outcome.
2. **Configuration-driven, not fixed.** Region scope, DR ratios, buffer targets, and scoring weights are all configurable (see the Configurable Items Register, Section 22 of the baseline). Hard-coded constants that are not in the register require an ADR.
3. **Cost is a primary constraint.** Idle reservation cost must be minimised. Minimal bootstrap + dynamic reconciliation is the governing model. Fixed large reserves require explicit business approval.
4. **Quota as a governor.** Quota is centrally governed. Individual teams may not independently acquire quota. The engine allocates quota on demand from the governed pool.

5. **Audit trail required.** Every capacity change, quota allocation, and placement decision must produce an auditable record (SOC 2 DR assertions are a compliance target).
 6. **Single-region failure assumption.** DR is designed for single-region failure (DR-001). Multi-region simultaneous failure is explicitly out of scope. The max-not-sum DR sizing follows directly from this assumption.
 7. **Preview features need production-readiness gates.** Capacity Reservation Sharing is still Preview. Any design that depends on it must be gated on DEP-001 (GA confirmation or explicit Microsoft supportability agreement).
 8. **No multi-cloud.** Rejected at ELT level. Do not propose multi-cloud alternatives.
-

Section 11 — Your Task

Define your task here before sending this prompt. Replace this section with a specific, scoped description of the research or design work required. Examples below show the expected format.

Example Task A — Technical Research

TASK: Research and document the current Azure Capacity Reservation Sharing feature (as of October 2026) against the ACRME design.

Specifically:

1. What is the current Preview status and expected GA timeline?
2. Does the consumer subscription require its own quota when consuming a shared reservation? (POC-001 / QUA-013)
3. What are the supported sharing topologies (cross-subscription, cross-resource-group, cross-region)?
4. What are the known limitations relevant to ACRME's multi-subscription DR model?

For each finding, tag confidence: [Documented], [Derived], [Assumed].
Flag any finding that contradicts or expands the current ACRME design.
Produce a structured findings report targeting 1000–1500 words.

Example Task B — Architecture Design

TASK: Design the Phase 3 Placement Engine component for ACRME.

Scope:

- Customer seed record creation and reuse (PLC-003, Scenario 20)
- CVAL/DR weighted placement and co-location (PLC-010a, PLC-010b)
- Hard constraint gate (HC-1..HC-10)
- Concurrent deployment control
- Capacity commitment workflow

Required outputs:

1. Component model (inputs, outputs, internal state)
2. API design (REST endpoints, request/response schema)
3. Data model (seed record, placement decision, commitment record)
4. Failure handling and rollback strategy
5. DR implications and WAF assessment (Security / Reliability / Performance / Cost / Operational Excellence)

Follow the 5-phase research methodology before designing:

Phase 0 (knowledge), Phase 1 (capacity research), Phase 2 (quota), Phase 3 (sharing), Phase 4 (POC definition), Phase 5 (architecture).

Example Task C — Formula Validation

TASK: Validate the PS_Prod scoring formula against the stated design principles and produce a sensitivity analysis.

Questions to answer:

1. Under what conditions does the γ (distribution fairness) term dominate the score and produce a non-intuitive result?
2. If `total_customers` is defined as "currently active customers in the geography", how does the score behave when a geography has 1 customer vs 100 customers?
3. What weight configuration produces the most stable ranking across the US three-region model?

Produce a worked numeric example for each question using the US three-region model (West US 3, Central US, Canada Central).

Tag all assumptions. Flag the `total_customers` spec gap explicitly.

WAF Assessment Template (Mandatory for Architecture Tasks)

Every architectural recommendation must be evaluated against all five WAF pillars. Include this structure in any architecture output:

Security

- Identity and RBAC requirements
- Managed Identity usage

- Key Vault / secret management
- Network isolation requirements
- Policy and governance controls

Reliability

- Availability Zone coverage
- Failure mode analysis (what happens when this component fails?)
- RTO / RPO implications
- Reconciliation and self-healing behaviour

Performance Efficiency

- Throughput and latency targets
- API rate limits (Azure control plane throttling is a known risk — POC-004)
- Scaling model

Cost Optimization

- Reservation idle cost implications
- Quota allocation cost implications
- FinOps tagging and tracking requirements

Operational Excellence

- Automation opportunities
- Monitoring and observability requirements
- Deployment model (IaC, CI/CD)
- Dry-run capability requirements

Anti-Patterns — Never Do These

1. **Do not assume undocumented Azure behaviour.** If it is not on Microsoft Learn or confirmed by a POC, tag it `[Assumed]` and flag it for validation.
2. **Do not skip quota validation.** Every capacity design has quota implications — they are not separable.
3. **Do not skip the WAF assessment.** Architecture outputs without WAF analysis are incomplete.
4. **Do not ignore DR implications.** Phase 1-only changes can have Phase 4 consequences.
5. **Do not treat Preview features as production-ready.** Capacity Reservation Sharing is Preview — gate any design on POC/GA confirmation.

6. **Do not generate production code before architecture is understood.** Follow the research → design → POC → implementation sequence.
 7. **Do not define `total_customers` without flagging it as a spec gap.** It has no authoritative definition in baseline v2.4.
 8. **Do not propose a fixed large DR reserve.** This has been rejected for cost reasons. Minimal bootstrap + dynamic reconciliation is the only approved DR model.
 9. **Do not propose multi-cloud.** Rejected at ELT level.
 10. **Do not modify the v2.4 baseline document directly.** Changes to baseline require an approved ADR and version increment.
-